Physics-Informed Neural Networks for Asian Options Pricing

Sara Borello

Data Science A.Y. 2024/2025

Supervisor : Matteo Pelagatti

Asian Option vs European Option

European Option

$$\underset{\text{call}}{C} = (\boxed{S_T} - K)^+$$

$$\underset{\text{put}}{P} = (K - \boxed{S_T})^+$$

where $(x)^+ = \max\{x, 0\}$

Path-independent payoff: depends only on the terminal price S_t

Asian Option

$$\underset{\text{call}}{C} = \left(\boxed{\frac{1}{T}\int_0^T S_u\,du} - K\right)^+$$

$$\underset{\text{put}}{P} = \left(K - \boxed{\frac{1}{T}\int_0^T S_u\,du}\right)^+$$

where $(x)^+ = \max\{x, 0\}$

Path-dependent payoff: the entire price trajectory [O,T] matters

Pricing Problem

Given the expected behavior of the underlying asset and the associated risk, what is a fair price for this option today?

Determining a function **V(t, S)** that:

- satisfies the Black–Scholes PDE
- admits an analytical closed-form solution.

In this case, **pricing reduces to the direct evaluation of an explicit formula.**

Determining a **function V(t,S,I)** that:

- satisfies a PDE/PIDE (jumps),
- does not admit an analytical closed-form solution.

Pricing therefore requires the numerical solution of using computational methods such as Monte Carlo

Motivation & Objective

Why Asian Options in Commodity Markets

- **Alignment with physical market conventions:** Asian options reflect average-based settlement practices common in electricity, oil, and metals markets
- **Robust hedging under volatility and jumps:** Averaging dampens short-lived spikes, seasonality, and jumps, yielding more stable hedging for spot-exposed producers and industrial users

Problem

Determining a **function V(t,S,I)** that:

- satisfies a PDE/PIDE (jumps),
- does not admit an analytical closed-form solution.

ACTUAL SOLUTION

Pricing therefore requires the numerical solution of using computational methods such as Monte Carlo

LIMITATIONS

- ❌ High computational cost
- ❌ Inefficient for calibration and real-time pricing

Final Goals

- **PINN-based pricing framework:** Develop a PINN to price Asian options by directly solving the associated PDE/PIDE, avoiding Monte Carlo simulation
- **Efficiency and consistency:** Enable fast, scalable repeated pricing while preserving the underlying stochastic dynamics and no-arbitrage conditions

Deep Learning for Option Pricing

PDE as the governing law

$$\mathcal{N}[u](x,t;\lambda) = 0$$

$$(x,t) \in \Omega \times (0,T]$$

Neural approximation

$$u(x,t) \approx \tilde{u}(x,t;\theta)$$

$$\tilde{u} : \Omega \times [0,T] \to \mathbb{R}$$

Key idea: PDE residual

(1) $$r(x,t;\theta) = \mathcal{N}[\tilde{u}(x,t;\theta)]$$

$$\mathcal{L}(\theta) = w_f \sum_{(x,t)\in T_f} r(x,t;\theta)^2 + w_b \sum_{(x,t)\in T_b} (\tilde{u} - g)^2 + w_i \sum_{x\in T_i} (\tilde{u} - h)^2$$

Phisical Informed Neural Network

PINN Methodology for Pricing Asian Options

Problem setting

Defining the PDE/PIDE for Asian Option

$$V = V(S, I, t) \quad \text{where} \quad I(t) = \int_0^t S(u)\,\mathrm{d}u$$

satisfies under pure diffusion dynamics the **PDE**

$$\partial_t V + rS\,\partial_S V + S\,\partial_I V + \tfrac{1}{2}\sigma^2 S^2\,\partial^2_{SS} V - rV = 0.$$

or satisfies **jump–diffusion process** with intensity λ and jump size distribution ν(y) the **PIDE**

$$\partial_t V + \tfrac{1}{2}\sigma^2 S^2 \partial^2_{SS} V + (r - \lambda\kappa) S \partial_S V + S \partial_I V - (r + \lambda) V + \boxed{\lambda \int_R V\left(S e^{\gamma(y)}, I, t\right) \nu(y)\,\mathrm{d}y} = 0,$$

jump

Strike normalization and scaling

variables normalized by the strike to improve numerical stability

$$S = \frac{S_{\text{mon}}}{K}, \qquad I = \frac{I_{\text{mon}}}{K}, \qquad V_n = \frac{V}{K}, \qquad \tau = T - t,$$

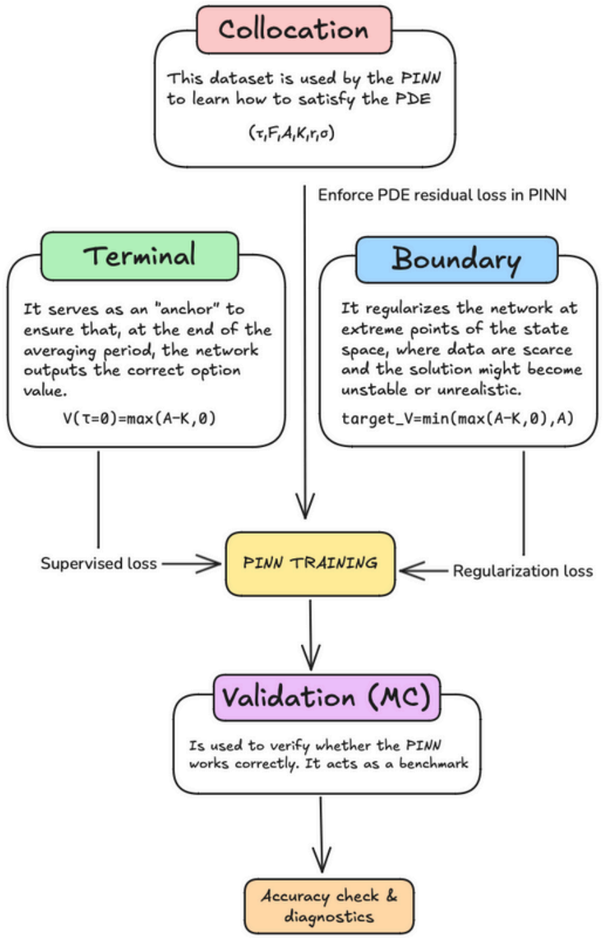
PINNs are trained assuming K = 1

Once the PINN learns the **normalized** solution, option values for any strike are recovered via a simple scaling relation.

$$V_{\text{mon}}(S', I', 0, r, \sigma; K') = K' \cdot V_n\left(\frac{S'}{K'}, \frac{I'}{K'}, 0, r, \sigma\right)$$

PINN Methodology for Pricing Asian Options

Overall schema and Training set up



The PINNs are trained to solve PDE/PIDE equations for one-asset Asian options **with K = 1 and T = 1,** considering the following ranges for the input variables:

$$S \in [0, S_{\max}], \quad I \in [0, I_{\max}], \quad r \in [0, r_{\max}], \quad \sigma \in [0, \sigma_{\max}]$$

Training domain ranges inferred from **WTI futures data**

Variable	Description	Real range	Normalized range
S	Spot price	[0, 148.44] USD	[0, 2.585]
I	Cumulative integral	[0, 148.44] USD	[0, 2.585]
r	Risk-free rate	[0, 0.50]	[0, 0.50]
σ	Volatility	[0, 0.807]	[0, 0.807]
T	Time to maturity	1.0 year	1.0
K	Strike price	57.42 USD	1.0

PINN Methodology for Pricing Asian Options

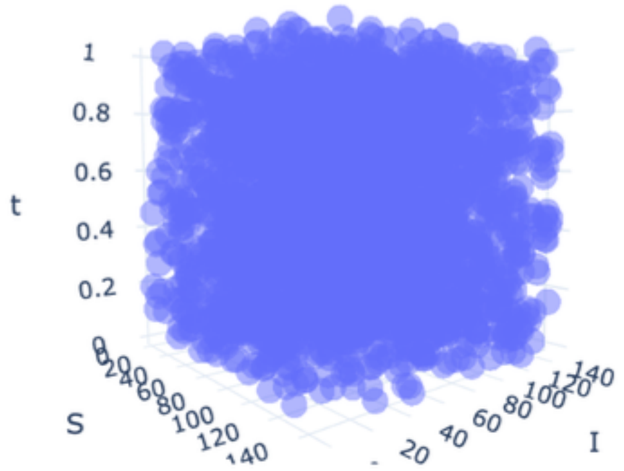
Data Construction for PINN Training: Collocation Points

Collocation points enforce the PDE by minimizing its residual inside the domain, without using market data

They are sampled from the normalized state space

1. sample $S \sim \mathcal{U}(0, S_{\max})$,
2. sample $I \sim \mathcal{U}(0, I_{\max})$,
3. sample $\tau \sim \mathcal{U}(0, 1)$,
4. sample $r \sim \mathcal{U}(0, r_{\max})$,
5. sample $\sigma \sim \mathcal{U}(0, \sigma_{\max})$,

concatenate the sampled variables to form the input vector (S, I, τ, r, σ)



Three-dimensional projection of sampled collocation points in the (S,I,t) space. The uniform distribution confirms homogeneous coverage of the interior domain.

PINN Methodology for Pricing Asian Options

Data Construction for PINN Training: Terminal Points

Terminal points anchor the solution at regions where the option value is known analytically or by construction

$$V_n(S, I, T, r, \sigma) = \max(I - 1, 0)$$

The terminal condition enforces the payoff structure of the Asian option at maturity. At t = T, the option value is fully determined by the cumulative integral variable and is independent of future dynamics

Algorithm 1 Sampling of Terminal Condition Dataset

Require: Number of terminal points n_{TC}; domain bounds $S_{\max}, I_{\max}, r_{\max}, \sigma_{\max}$

Ensure: Terminal condition dataset $(\mathbf{X}_{\mathrm{TC}}, \mathbf{Y}_{\mathrm{TC}})$

for $i = 1, \ldots, n_{\mathrm{TC}}$ **do**

  Sample spot price $S_i \sim \mathcal{U}(0, S_{\max})$

  Sample cumulative integral $I_i \sim \mathcal{U}(0, I_{\max})$

  Set time to maturity $t_i \leftarrow 1$

  Sample risk-free rate $r_i \sim \mathcal{U}(0, r_{\max})$

  Sample volatility $\sigma_i \sim \mathcal{U}(0, \sigma_{\max})$

  Construct input vector

$$\mathbf{x}_i = (S_i, I_i, t_i, r_i, \sigma_i)$$

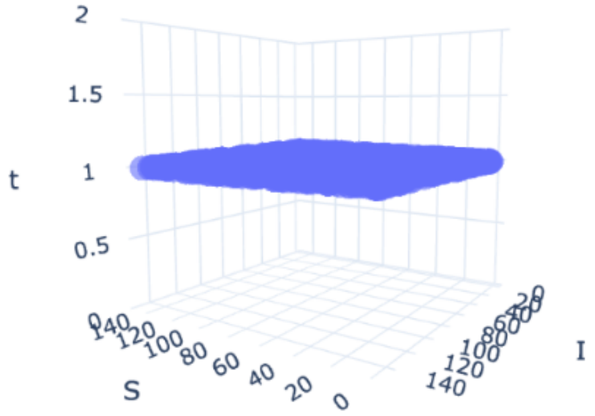
  Compute target payoff

$$y_i = \max(I_i - 1, 0)$$

end for

Assemble dataset

$$\mathbf{X}_{\mathrm{TC}} = \{\mathbf{x}_i\}_{i=1}^{n_{\mathrm{TC}}}, \qquad \mathbf{Y}_{\mathrm{TC}} = \{y_i\}_{i=1}^{n_{\mathrm{TC}}}$$



Terminal condition collocation points in the (S,I,t) space. All points lie on the hyperplane t = 1, corresponding to option maturity

PINN Methodology for Pricing Asian Options

Data Construction for PINN Training: Boundary Conditions

$$V_n(0, I, t, r, \sigma) \approx \mathrm{e}^{-r(1-t)} \max(I-1, 0)$$

The boundary condition at S=0 captures the degenerate case where the **underlying price remains zero,** so the option value depends only on the accumulated average I and the remaining time to maturity

$$V_n(S_{\max}, I, t, r, \sigma) \approx \mathrm{e}^{-r(1-t)}(I-1) + S_{\max}\frac{1-\mathrm{e}^{-r(1-t)}}{r}$$

At S=Smax, the **Asian call is deeply in the money** and the boundary condition approximates the option value as the discounted intrinsic value of the average plus a spot-price carry term, enforced by fixing S=Smax and sampling the remaining variables

$$V_n(S, I_{\max}, t, r, \sigma) \approx \mathrm{e}^{-r(1-t)}(I_{\max}-1) + S\frac{1-\mathrm{e}^{-r(1-t)}}{r}$$

At I=Imax, the **accumulated average dominates the payoff, so the Asian option is effectively in the money** and its value is approximated by the discounted intrinsic value of the average plus a spot-price carry term, enforced by fixing I=Imax and sampling the remaining variables

PINN Methodology for Pricing Asian Options

Data Construction for PINN Validation: Montecarlo

Algorithm 5 Monte Carlo Pricing of an Arithmetic Asian Call Option

Require: Initial spot price S_0, strike K, risk-free rate r, volatility σ, maturity T
Require: Number of time steps n_{steps}, number of paths n_{paths}, antithetic variates flag
Ensure: Monte Carlo price estimate and standard error

```
Set Δt ← T/n_steps
Compute discount factor e^{-rT}
if antithetic variates enabled then
    Sample n_paths/2 standard normal increments and construct antithetic pairs
else
    Sample n_paths standard normal increments
end if
Initialize S_0^(i) = S_0 and cumulative sums ∑ S^(i) = 0
for k = 1, ..., n_steps do
    Update prices:
```

$$S^{(i)} \leftarrow S^{(i)} \exp\left((r - \tfrac{1}{2}\sigma^2)\Delta t + \sigma\sqrt{\Delta t} Z_k^{(i)}\right)$$

```
    Accumulate prices
end for
Compute arithmetic averages and discounted payoffs
Estimate option price as the mean discounted payoff
Estimate the standard error of the estimator
```

PDE setting

Algorithm 6 Monte Carlo Pricing of an Arithmetic Asian Call under Jump–Diffusion

Require: Initial price S_0, strike K, rate r, volatility σ, maturity T
Require: Jump parameters $(\lambda, \mu_J, \sigma_J)$
Require: Number of paths N, number of time steps N_t
Ensure: Monte Carlo price estimate $\widehat{V}_{\text{MC}}$

```
Set time step Δt ← T/N_t
Compute compensator κ ← exp(μ_J + ½σ_J²) − 1
Initialize S^(i) ← S_0, A^(i) ← 0 for i = 1, ..., N
for k = 1 to N_t do
    for i = 1 to N do
        Sample Z_k^(i) ~ N(0,1)
        Sample N_k^(i) ~ Poisson(λΔt)
        Update diffusion:
```

$$S^{(i)} \leftarrow S^{(i)} \exp\left[(r - \lambda\kappa - \tfrac{1}{2}\sigma^2)\Delta t + \sigma\sqrt{\Delta t}\, Z_k^{(i)}\right]$$

```
        if N_k^(i) > 0 then
            Sample Y^(i) ~ N(μ_J, σ_J²)
            Apply jump: S^(i) ← S^(i) e^{Y^(i)}
        end if
        Accumulate average: A^(i) ← A^(i) + S^(i)
    end for
end for
Compute arithmetic average A_T^(i) ← A^(i)/N_t
Compute payoff P^(i) ← max(A_T^(i) − K, 0)
Discount and average:
```

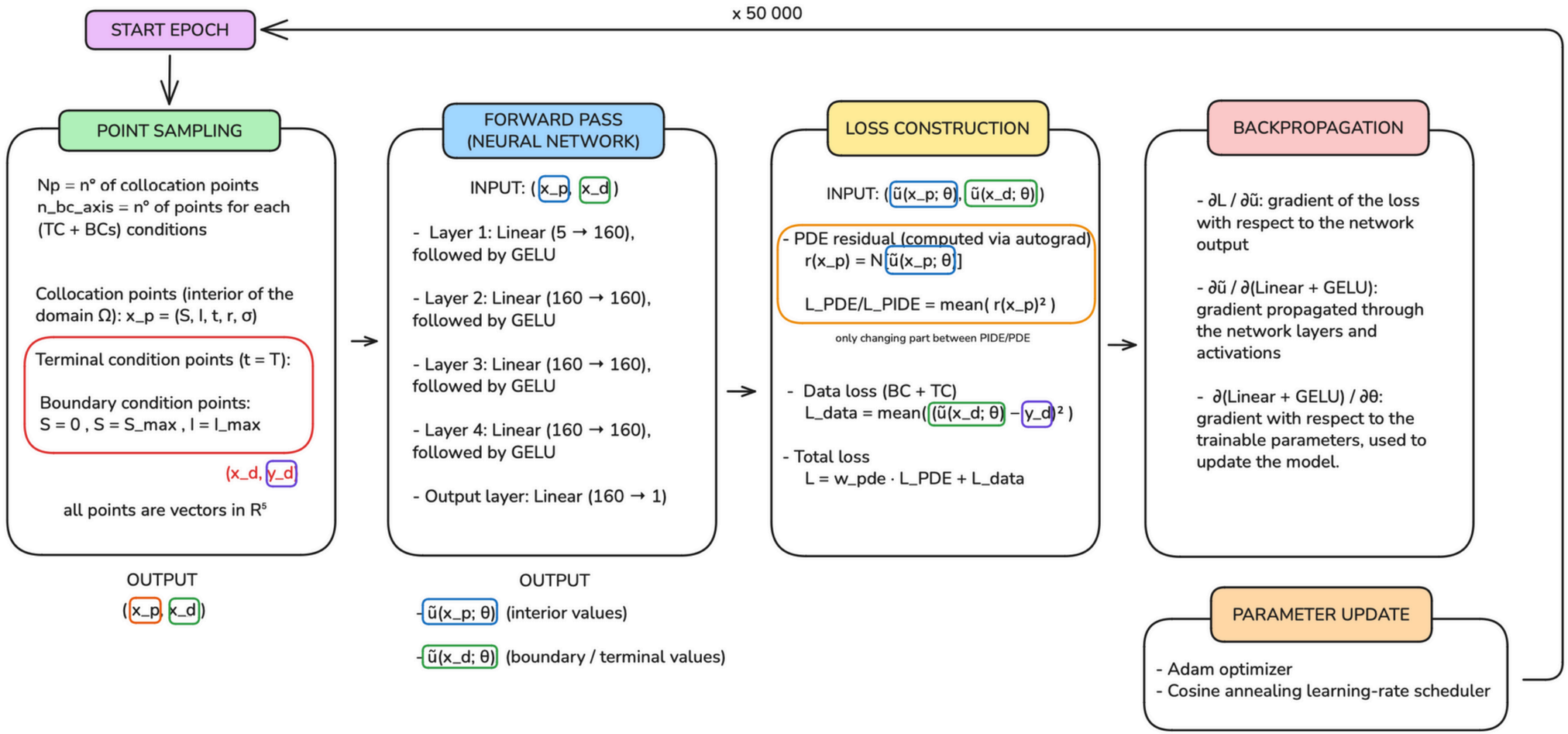
$$\widehat{V}_{\text{MC}} = e^{-rT} \frac{1}{N} \sum_{i=1}^{N} P^{(i)}$$

```
return V̂_MC
```

PIDE setting

PINN Methodology for Pricing Asian Options

Neural Network building general



PINN Methodology for Pricing Asian Options

Neural Network building: PDE LOSS

Algorithm 7 PDE Residual Loss Computation

Require: Collocation points $X = (S, I, t, r, \sigma)$

Ensure: PDE loss ℓ_{PDE}

1: Normalize S and I; keep t, r, σ in physical units

2: Enable gradient tracking: $X \leftarrow \texttt{requires_grad}(X)$

3: Compute network output: $V \leftarrow V_\theta(X)$

4: Compute first-order derivatives via automatic differentiation:

$$\nabla_X V = (\partial_S V,\ \partial_I V,\ \partial_t V,\ \partial_r V,\ \partial_\sigma V)$$

5: Extract required gradients:

$$V_S = \partial_S V, \quad V_I = \partial_I V, \quad V_t = \partial_t V$$

6: Compute second-order derivative with respect to S:

$$V_{SS} = \partial_{SS} V$$

7: Assemble the PDE residual:

$$\text{res} = V_t + \frac{1}{2}\sigma^2 S^2 V_{SS} + rSV_S + SV_I - rV$$

8: Compute the PDE loss:

$$\ell_{\text{PDE}} = \frac{1}{N_{\text{PDE}}} \sum_{i=1}^{N_{\text{PDE}}} \text{res}_i^2$$

return ℓ_{PDE}

- PyTorch supports PINNs through a dynamic computational graph and reverse-mode automatic differentiation
- By enabling gradient tracking on the inputs, it allows consistent computation of first- and second-order derivatives without finite-difference approximations, yielding an exact PDE residual up to machine precision

PINN Methodology for Pricing Asian Options

Neural Network building: PIDE LOSS

Algorithm 8 Computation of the PIDE Residual for an Arithmetic Asian Option under Jump–Diffusion

Require: Neural network model V_θ, input tensor $X = (S, I, t, r, \sigma)$

Ensure: PIDE residual $\mathcal{R}_{\text{PIDE}}(X)$

Set jump–diffusion parameters (Merton model):

Jump intensity λ, jump mean μ_J, jump volatility σ_J

Compute jump compensator:

$$\kappa = E\left[e^Y - 1\right], \qquad Y \sim \mathcal{N}(\mu_J, \sigma_J^2)$$

Initialize Gauss–Hermite quadrature:

Choose the number of nodes n_{GH}

Obtain quadrature nodes $\{y_i\}_{i=1}^{n_{\text{GH}}}$ and weights $\{w_i\}_{i=1}^{n_{\text{GH}}}$

Rescale quadrature nodes:

for $i = 1$ to n_{GH} **do**

$$y_i \leftarrow \sqrt{2}\,\sigma_J y_i + \mu_J, \qquad w_i \leftarrow \frac{w_i}{\sqrt{\pi}}$$

end for

Enable automatic differentiation on inputs

Evaluate network output $V \leftarrow V_\theta(X)$

The **jump component is modeled using the Merton jump–diffusion framework**, where jumps arrive according to a **Poisson process** with constant intensity λ and logarithmic jump sizes follow a Gaussian distribution with fixed parameters controlling the average magnitude and dispersion of jumps

Compute first-order derivatives:

$$V_S = \frac{\partial V}{\partial S}, \qquad V_I = \frac{\partial V}{\partial I}, \qquad V_t = \frac{\partial V}{\partial t}$$

Compute second-order derivative:

$$V_{SS} = \frac{\partial^2 V}{\partial S^2}$$

Approximate jump integral via quadrature:

Initialize

$$\mathcal{J} = 0$$

for $i = 1$ to n_{GH} **do**

$$S^{(i)} = S\exp(y_i), \qquad X^{(i)} = \left(S^{(i)}, I, t, r, \sigma\right)$$

$$\mathcal{J} \leftarrow \mathcal{J} + w_i V_\theta\left(X^{(i)}\right)$$

end for

Assemble the PIDE residual:

$$\boxed{\mathcal{R}_{\text{PIDE}} = V_t + \frac{1}{2}\sigma^2 S^2 V_{SS} + (r - \lambda\kappa) S V_S + S V_I - (r + \lambda) V + \lambda \mathcal{J}}$$

Numerical Experiments: PDE Epoch Sensitivity

Parameters Settings

Table 5.1: Fixed training configuration used in the epoch sensitivity experiments.

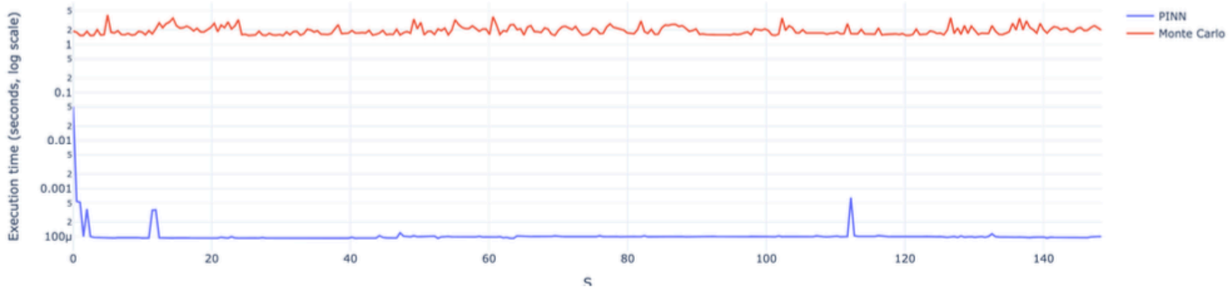
Category	Parameter	Value
Architecture	Input dimension n	5
	Hidden layer width N	160
	Network depth d	4
Optimization	Optimizer	Adam
	Initial learning rate η_0	10^{-3}
	Learning rate schedule	Cosine annealing (to zero)
Training setup	PDE collocation points N_{PDE}	1,000 per epoch
	Boundary / terminal samples n_{BC}	100 per axis
	PDE loss weight w_{PDE}	1.0
	Underlying domain	WTI-calibrated

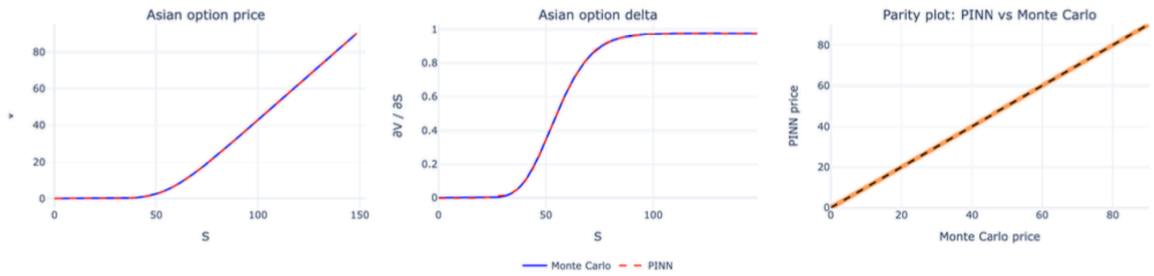
Assessment between models

Table 5.5: Comparison of convergence and evaluation results across different epochs

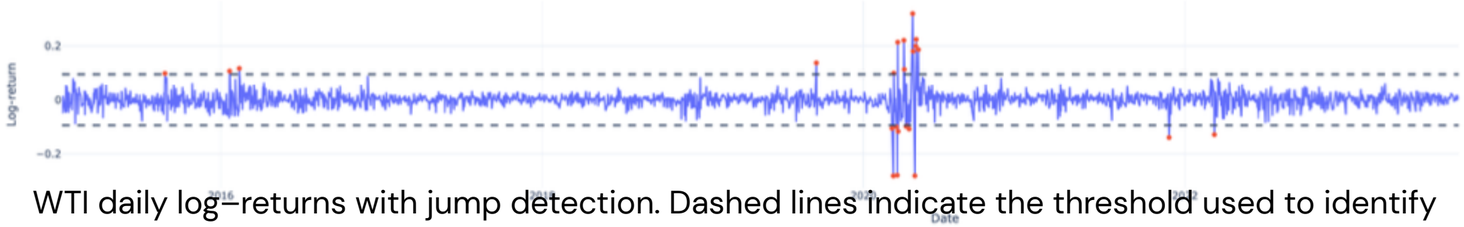
Metric	5,000 epochs	20,000 epochs	50,000 epochs
Total training time [min]	6.57	16.47	30.8
Final total loss	2.772×10^{-4}	2.059×10^{-5}	3.58×10^{-6}
Final PDE loss	1.466×10^{-4}	9.825×10^{-5}	1.93×10^{-6}
Final data loss	1.306×10^{-4}	1.077×10^{-5}	1.66×10^{-6}
MAE (PINN vs MC)	0.7935	7.419×10^{-2}	1.91×10^{-2}
Speed-up (MC / PINN)	1.995×10^{4}	1.988×10^{4}	1.96×10^{4}

Best Models Performance: 50,000 epochs





Numerical Experiments: PINN for PIDE



WTI daily log–returns with jump detection. Dashed lines indicate the threshold used to identify jump events, while red markers denote detected jumps

Jump events are detected using a threshold rule, classifying as jumps all log–returns whose absolute value exceeds

$$C\sigma\sqrt{\Delta t} \quad \text{with } C = 4$$

Parameters settings

Table 5.1: Fixed training configuration used in the epoch sensitivity experiments.

Category	Parameter	Value
Architecture	Input dimension n	5
	Hidden layer width N	160
	Network depth d	4
Optimization	Optimizer	Adam
	Initial learning rate η_0	10^{-3}
	Learning rate schedule	Cosine annealing (to zero)
Training setup	PDE collocation points N_{PDE}	1,000 per epoch
	Boundary / terminal samples n_{BC}	100 per axis
	PDE loss weight w_{PDE}	1.0
	Underlying domain	WTI-calibrated

λ (jumps/year)	2.188
μ_J	0.0196
σ_J	0.1817
κ	0.0368

Calibrated jump–diffusion parameters for WTI crude oil futures

Metric	$epochs = 10000$	$epochs = 20000$	$epochs = 50000$
Total training time (min)	15.17	28.11	63
Final total loss	5.450×10^{-5}	1.481×10^{-5}	4.682×10^{-6}
Final PDE loss	5.309×10^{-5}	1.289×10^{-5}	2.087×10^{-6}
Final data loss	2.264×10^{-5}	7.081×10^{-5}	3.430×10^{-6}
MAE (PINN vs MC)	0.25	0.0718	0.03182
Speed-up (MC / PINN)	8.486×10^4	3.0021×10^4	1.914×10^4

Results

Conclusions

Summary of Findings

- PINNs accurately price arithmetic Asian options calibrated to WTI futures, matching Monte Carlo accuracy under diffusion dynamics with large computational speed-ups
- Extensions to jump–diffusion models are feasible but introduce higher errors due to PIDE complexity, leading to a clear cost–accuracy trade-off

Implications

- PINNs can act as fast surrogate pricing engines for path-dependent derivatives, enabling efficient pricing and Greeks computation in practical settings
- PINNs naturally extend from PDE to PIDE formulations, supporting nonlocal dynamics while remaining complementary to Monte Carlo methods.

Limitations and Future Work

- The analysis relies on synthetic benchmarks and limited model classes, motivating extensions to real market data and richer stochastic dynamics
- Future research should explore improved architectures, adaptive sampling, and more accurate treatments of nonlocal jump integrals